

## QUESTION 1 BONUS

*Who are the Top 10 customers by total amount spent?*

Query & Result Screenshot:

The screenshot displays a database IDE interface with a dark theme. On the left, the 'Database Explorer' pane shows a tree structure of a database named 'MySQLLaptop', including schemas like 'information\_schema', 'mydatabase', 'mysql', 'netticks', and 'olist'. The 'olist' schema is expanded, showing tables such as 'customers', 'geolocation', 'order\_items', 'order\_payments', 'order\_reviews', 'orders', 'products', and 'sellers'. The main editor pane shows a SQL query for 'Q1' with the following text:

```
-- Who are the Top 10 customers by total amount spent?
#select * from customers;
#select * from orders;
#select * from order_payments;

use olist;

select
  c.customer_id,
  sum(op.payment_value) as total_spent
from customers as c
join orders as o on c.customer_id = o.customer_id
join order_payments as op on o.order_id = op.order_id
group by c.customer_id
order by total_spent desc
limit 10;
```

Below the editor, the 'Services' pane is open, showing the 'Output' tab. It displays the results of the query, titled 'Capstone Q1 Result'. The results are shown as a table with 10 rows and 2 columns: 'customer\_id' and 'total\_spent'.

| customer_id                      | total_spent |
|----------------------------------|-------------|
| 1617b1357756262bfa5aab541c47bc10 | 13666.88    |
| ec5b2baa2e574342386871631fafd3fc | 7274.88     |
| c6e2751c5b391845f6800c974d1a43e9 | 6929.31     |
| f48d4e4a0b8aea338cb25f810991ab1f | 6922.21     |
| 3fd6777bbce88a352fdd004e4a7cc8f6 | 6726.66     |
| 05455dfa7cd02f13d132aa7a6a9729c6 | 6081.54     |
| df55c14d1470a9a3467f131269c2477f | 4950.34     |
| e0a2412720e9eakf2c1ac985fca7358  | 4809.44     |
| 24bbf5f62f2e1b359ee7de94defc4a15 | 4764.34     |
| 3d979689f636322c62418b6346b1cd2  | 4681.78     |

The bottom status bar indicates the current file is 'Q1 [MySQLLaptop]', the cursor is at line 29, column 11, and the encoding is UTF-8.

## QUESTION 2

*What is the monthly revenue trend across the dataset?*

### Query & Result Screenshot:

The screenshot displays a MySQL IDE interface. On the left, the 'Database Explorer' shows a schema named 'mydatabase' with tables 'customers', 'geolocation', 'order\_items', 'order\_payments', and 'order\_reviews'. The 'Services' panel on the left lists various services, including 'Q1', 'Q2', 'Q3', 'Q4', 'Q5', 'Q6', 'Q7', 'Q8', 'console\_1', 'console\_2', and 'help'. The main editor shows a SQL query in a file named 'Q2 [MySQLLaptop]'. The query is as follows:

```
-- What is the monthly revenue trend across the dataset?

use olist;

SELECT
    DATE_FORMAT(o.order_purchase_timestamp, '%b %Y') AS month,
    CONCAT('$', FORMAT(SUM(op.payment_value), 2)) AS monthly_revenue
FROM orders AS o
JOIN order_payments AS op
    ON o.order_id = op.order_id
GROUP BY DATE_FORMAT(o.order_purchase_timestamp, '%b %Y')
ORDER BY MIN(o.order_purchase_timestamp);
```

The 'Output' panel on the right shows the results of the query, which is a table with two columns: 'month' and 'monthly\_revenue'. The results are as follows:

| month       | monthly_revenue |
|-------------|-----------------|
| 1 Sep 2016  | \$252.24        |
| 2 Oct 2016  | \$59,890.48     |
| 3 Dec 2016  | \$19.62         |
| 4 Jan 2017  | \$138,488.04    |
| 5 Feb 2017  | \$291,908.01    |
| 6 Mar 2017  | \$449,863.60    |
| 7 Apr 2017  | \$417,788.03    |
| 8 May 2017  | \$592,918.82    |
| 9 Jun 2017  | \$511,276.38    |
| 10 Jul 2017 | \$592,382.92    |
| 11 Aug 2017 | \$674,396.32    |
| 12 Sep 2017 | \$727,762.45    |
| 13 Oct 2017 | \$778,677.88    |
| 14 Nov 2017 | \$1,194,882.80  |
| 15 Dec 2017 | \$878,401.48    |
| 16 Jan 2018 | \$1,115,004.18  |
| 17 Feb 2018 | \$992,463.34    |

The bottom status bar indicates the file encoding is UTF-8, the line ending is CRLF, and there are 4 spaces. A 'Non-commercial use' license notice is also visible.

## QUESTION 3

What is the month-over-month change in revenue?

### Query & Result Screenshot:

The screenshot displays a MySQL IDE interface with a dark theme. On the left, the 'Database Explorer' pane shows a tree view of the 'MySQLLaptop' database, including schemas like 'information\_schema', 'mysql', and 'sakila'. The main editor pane contains a SQL query (Q3) that calculates the month-over-month change in revenue. The query uses a CTE named 'monthly\_revenue' to aggregate revenue by month, then uses a window function to calculate the percentage change between consecutive months. The 'Services' pane at the bottom shows the execution time for various queries. The 'Output' pane displays the results of the query, showing a table with columns 'month', 'revenue', and 'mom\_change'.

```
1 -- What is the month-over-month change in revenue?
2 WITH monthly_revenue AS (
3     SELECT
4         DATE_FORMAT(o.order_purchase_timestamp, '%b %Y') AS month,
5         DATE_FORMAT(o.order_purchase_timestamp, '%Y-%m') AS month_sort,
6         SUM(op.payment_value) AS revenue
7     FROM orders AS o
8     JOIN order_payments AS op
9         ON o.order_id = op.order_id
10    GROUP BY
11        DATE_FORMAT(o.order_purchase_timestamp, '%b %Y'),
12        DATE_FORMAT(o.order_purchase_timestamp, '%Y-%m')
13 )
14
15 SELECT
16     month,
17     CONCAT('$', FORMAT(revenue, 2)) AS revenue,
18     CONCAT(
19         ROUND(
20             (revenue - LAG(revenue) OVER (ORDER BY month_sort))
21             / LAG(revenue) OVER (ORDER BY month_sort) * 100,
22             2
23         ),
24         '%'
25     ) AS mom_change
26 FROM monthly_revenue
27 ORDER BY month_sort;
```

| month    | revenue      | mom_change |
|----------|--------------|------------|
| Sep 2016 | \$252.24     | <null>     |
| Oct 2016 | \$59,090.48  | 23326.29%  |
| Dec 2016 | \$19.62      | -99.97%    |
| Jan 2017 | \$138,488.04 | 705751.38% |
| Feb 2017 | \$291,908.01 | 110.78%    |

## QUESTION 4

*Which product categories generate the most revenue?*

**Query & Result Screenshot:**

The screenshot displays a database IDE interface with a dark theme. The top panel shows a SQL query in a tab labeled 'Q4 [MySQLLaptop]'. The query is as follows:

```
1 -- Which product categories generate the most revenue?
2
3 SELECT
4     p.product_category_name AS product_category,
5     CONCAT(' ', FORMAT(SUM(o1.price), 2)) AS total_revenue
6 FROM order_items AS o1
7 JOIN products AS p
8     ON o1.product_id = p.product_id
9 GROUP BY
10     p.product_category_name
11 ORDER BY
12     SUM(o1.price) DESC;
```

The bottom panel shows the results of the query in a table with two columns: 'product\_category' and 'total\_revenue'. The table contains 15 rows of data, sorted by total revenue in descending order. The first row is 'beleza\_saude' with a total revenue of \$1,258,681.34. The last row is 'pcs' with a total revenue of \$222,963.13. The table indicates 74 rows in total.

| product_category       | total_revenue  |
|------------------------|----------------|
| beleza_saude           | \$1,258,681.34 |
| relorios_presentes     | \$1,205,005.68 |
| cama_mesa_banho        | \$1,036,988.68 |
| esporte_lazer          | \$988,848.97   |
| informatica_acessorios | \$911,954.32   |
| moveis_decoracao       | \$729,762.49   |
| cool_stuff             | \$635,290.85   |
| utilidades_domesticas  | \$632,268.66   |
| automotivo             | \$592,720.11   |
| ferramentas_jardim     | \$485,256.46   |
| brinquedos             | \$483,946.68   |
| bebes                  | \$411,704.89   |
| perfumaria             | \$399,124.87   |
| telefonica             | \$323,667.53   |
| moveis_escritorio      | \$273,960.70   |
| papelaria              | \$230,943.23   |
| pcs                    | \$222,963.13   |

## QUESTION 5

Rank the Top 3 Products within each category by revenue.

### Query & Result Screenshot:

The screenshot displays a MySQL IDE interface with a query editor and a results pane. The query aims to rank the top 3 products within each category by revenue.

```
-- Rank the Top 3 Products within each category by revenue.

WITH product_revenue AS (
  SELECT
    p.product_category_name AS product_category,
    oi.product_id,
    SUM(oi.price) AS total_revenue
  FROM order_items AS oi
  JOIN products AS p
    ON oi.product_id = p.product_id
  GROUP BY
    p.product_category_name,
    oi.product_id
),
ranked_products AS (
  SELECT
    product_category,
    product_id,
    total_revenue,
    ROW_NUMBER() OVER (
      PARTITION BY product_category
      ORDER BY total_revenue DESC
    ) AS rn
  FROM product_revenue
)
SELECT
  product_category,
  product_id,
  CONCAT('$', FORMAT(total_revenue, 2)) AS total_revenue,
  rn
FROM ranked_products
WHERE rn <= 3
ORDER BY
  product_category,
  rn;
```

The results pane shows the output of the query, displaying the top 3 products by revenue for each category. The results are as follows:

| product_category          | product_id                        | total_revenue | rn |
|---------------------------|-----------------------------------|---------------|----|
| <null>                    | 5a848e4ab52f45445cdc07aabb1c40e48 | \$24,229.03   | 1  |
| <null>                    | eed5cd874fac3bd779c76c95fa7507d   | \$9,945.00    | 2  |
| <null>                    | b1d20758fca400a2378d50a9ba1da98   | \$7,152.00    | 3  |
| agro_industria_e_comercio | 11250b0d4b709fee92441c3f34122aed  | \$9,111.00    | 1  |
| agro_industria_e_comercio | 423a6644fbaa52e8828ff1f9100369d   | \$8,043.00    | 2  |
| agro_industria_e_comercio | 672e757f33190b9fdeea127a2a7b79fd  | \$6,885.00    | 3  |
| alimentos                 | 732a6828aa5efe1ba096223de496f596  | \$4,461.02    | 1  |
| alimentos                 | 89321f94e35fc6d7903d36f74e351d40  | \$3,375.71    | 2  |
| alimentos                 | ed2067a9c1f79553088a3c67b99a9f97  | \$3,187.50    | 3  |
| alimentos_bebidas         | 992197904e1d4f0bf3994652373188e4  | \$2,586.72    | 1  |
| alimentos_bebidas         | 90f97298579c20412f4dc9b7a2d4b0b   | \$1,348.00    | 2  |
| alimentos_bebidas         | 84f5c4f480a6dc99986a6a8a0f1a2e41  | \$987.96      | 3  |
| artes                     | 4fe644d7b6c756d6bca4fba851363cb3b | \$10,803.72   | 1  |
| artes                     | 1bd5e6731585cf01aa8169c7028d6ad   | \$6,499.00    | 2  |
| artes                     | cbf96c042050c933089e025748c2a057  | \$487.50      | 3  |
| artes_e_artisanato        | b9970e9c22fb1546bd71d1bcd2989475  | \$641.45      | 1  |
| artes_e_artisanato        | 6bb18295cc019bf3b0dc7c773411d530  | \$289.49      | 2  |
| artes_e_artisanato        | 4350dd2b0b5e0535079f88c7dc51c4de  | \$238.00      | 3  |
| artigos_de_festas         | 99b459ab82b0f75d453108c9c75e01f4  | \$772.80      | 1  |
| artigos_de_festas         | 833f0d4f7b1c82365c8273e02de0c19f  | \$571.20      | 2  |
| artigos_de_festas         | ded0fc3d15300415b27c4b3342feef1   | \$398.00      | 3  |
| artigos_de_natal          | 40e8b423d1a2a6e269c77353523e05ce  | \$1,737.80    | 1  |
| artigos_de_natal          | f3cdd9f9c7f731e0ef6626e911        | \$1,052.10    | 2  |
| artigos_de_natal          | 9defb0bd3c270b0c41d1c8f707000000  | \$671.00      | 3  |

## QUESTION 6

Segment customers into spend tiers (Low/Medium/High).

### Query & Result Screenshot:

The screenshot displays a SQL IDE interface with a query editor and a results pane. The query editor shows a SQL query that segments customers into spend tiers based on their total spending. The results pane shows the output of the query, which is a table with three columns: customer\_id, total\_spent, and spend\_tier. The results are sorted by total\_spent in descending order.

```
1 -- Segment customers into spend tiers (Low/Medium/High).
2
3 WITH customer_spending AS (
4     SELECT
5         c.customer_id,
6         SUM(op.payment_value) AS total_spent
7     FROM customers AS c
8     JOIN orders AS o
9         ON c.customer_id = o.customer_id
10    JOIN order_payments AS op
11        ON o.order_id = op.order_id
12    GROUP BY c.customer_id
13 )
14 SELECT
15     customer_id,
16     CONCAT('$', FORMAT(total_spent, 2)) AS total_spent,
17     CASE
18         WHEN total_spent < 100 THEN 'Low'
19         WHEN total_spent < 500 THEN 'Medium'
20         ELSE 'High'
21     END AS spend_tier
22 FROM customer_spending
23 ORDER BY total_spent DESC;
```

| customer_id                      | total_spent | spend_tier |
|----------------------------------|-------------|------------|
| ba7624c0778d41f67dc8e228df61267e | \$991.20    | High       |
| 74920bc93e00ed778ab3be3f83120995 | \$991.09    | High       |
| 0929b20c338c09f8a70e85340a3fb433 | \$990.54    | High       |
| 7199038790940ae4e75b8efc880f9dd2 | \$990.45    | High       |
| b0c90e175370cde72e1640bf58485d24 | \$99.99     | Low        |
| 28791c94579a4d9701f4c05eac5f87de | \$99.98     | Low        |
| 1b49d294f870ff4d42d0e0a33b040e05 | \$99.96     | Low        |
| 01122215d021ac872ae567ec4e351e01 | \$99.95     | Low        |

## QUESTION 7

*How many customers are repeat buyers vs one-time buyers?*

### Query & Result Screenshot:

The screenshot displays a database IDE interface with a dark theme. On the left, the 'Database Explorer' pane shows a tree view of a MySQL database named 'MySQLLaptop'. The tree includes schemas like 'information\_schema', 'mydatabase', 'myd', 'myfficks', and 'olist', along with various tables and views. The main editor pane shows a SQL query labeled 'Q7 [MySQLLaptop]'. The query is designed to identify repeat buyers versus one-time buyers by analyzing the 'customers' and 'orders' tables. It uses a UNION ALL to combine two queries: one for repeat buyers (customers with more than one order) and one for one-time buyers (customers with exactly one order). The results are formatted as a table with columns 'buyer\_type' and 'customer\_count'. The 'Output' pane at the bottom shows the execution results for 'Q7 Result', displaying two rows: 'Repeat Buyer' with a count of 2,997 and 'One-Time Buyer' with a count of 93,099. The status bar at the bottom indicates the database is 'MySQLLaptop', the query is 'Q7 [MySQLLaptop]', and the encoding is 'UTF-8'.

```
1 -- How many customers are repeat buyers vs one-time buyers?
2
3 SELECT
4     'Repeat Buyer' AS buyer_type,
5     FORMAT(COUNT(*), 0) AS customer_count
6 FROM (
7     SELECT
8         c.customer_unique_id
9     FROM customers AS c
10    JOIN orders AS o
11      ON c.customer_id = o.customer_id
12    GROUP BY c.customer_unique_id
13    HAVING COUNT(o.order_id) > 1
14 ) AS repeat_customers
15
16 UNION ALL
17
18 SELECT
19     'One-Time Buyer' AS buyer_type,
20     FORMAT(COUNT(*), 0) AS customer_count
21 FROM (
22     SELECT
23         c.customer_unique_id
24     FROM customers AS c
25    JOIN orders AS o
26      ON c.customer_id = o.customer_id
27    GROUP BY c.customer_unique_id
28    HAVING COUNT(o.order_id) = 1
29 ) AS one_time_customers;
```

| buyer_type     | customer_count |
|----------------|----------------|
| Repeat Buyer   | 2,997          |
| One-Time Buyer | 93,099         |

## QUESTION 8

*What percent of total revenue comes from the top category?*

### Query & Result Screenshot:

The screenshot displays a database IDE interface with a dark theme. On the left, the 'Database Explorer' pane shows a tree view of a MySQL database named 'MySQLLaptop'. The tree includes folders for 'information\_schema', 'mydatabase', 'mysql', 'netflix', 'olist', 'performance\_schema', and 'sakila'. The 'olist' folder is expanded, showing a 'tables' sub-folder with various tables like 'customers', 'geolocation', 'order\_items', 'order\_payments', 'order\_reviews', 'orders', 'products', 'sellers', and 'performance\_schema'. The main editor pane shows a SQL query for 'Q8 [MySQLLaptop]'. The query is as follows:

```
-- What percent of total revenue comes from the top category?

SELECT
  p.product_category_name AS product_category,
  CONCAT('$', FORMAT(SUM(o1.price), 2)) AS category_revenue,
  CONCAT(
    ROUND(
      SUM(o1.price) / SUM(SUM(o1.price)) OVER () * 100,
      2
    ),
    '%'
  ) AS percent_of_total
FROM order_items AS o1
JOIN products AS p
  ON o1.product_id = p.product_id
GROUP BY p.product_category_name
ORDER BY SUM(o1.price) DESC
LIMIT 1;
```

Below the query editor, the 'Services' pane is visible. It contains a 'Tx' pane on the left showing a list of transactions, with 'Q8' selected. The 'Output' pane on the right shows the results of the query. The results are displayed in a table with three columns: 'product\_category', 'category\_revenue', and 'percent\_of\_total'. The first row shows the results for the top category, 'beleza\_saude'.

| product_category | category_revenue | percent_of_total |
|------------------|------------------|------------------|
| beleza_saude     | \$1,258,681.36   | 9.26%            |

The bottom status bar of the IDE shows the current file is 'MySQLLaptop > Q8 [MySQLLaptop]' and the encoding is 'UTF-8'.